다차원 배열 객체

이청용 교수
leecy@hansung.ac.kr

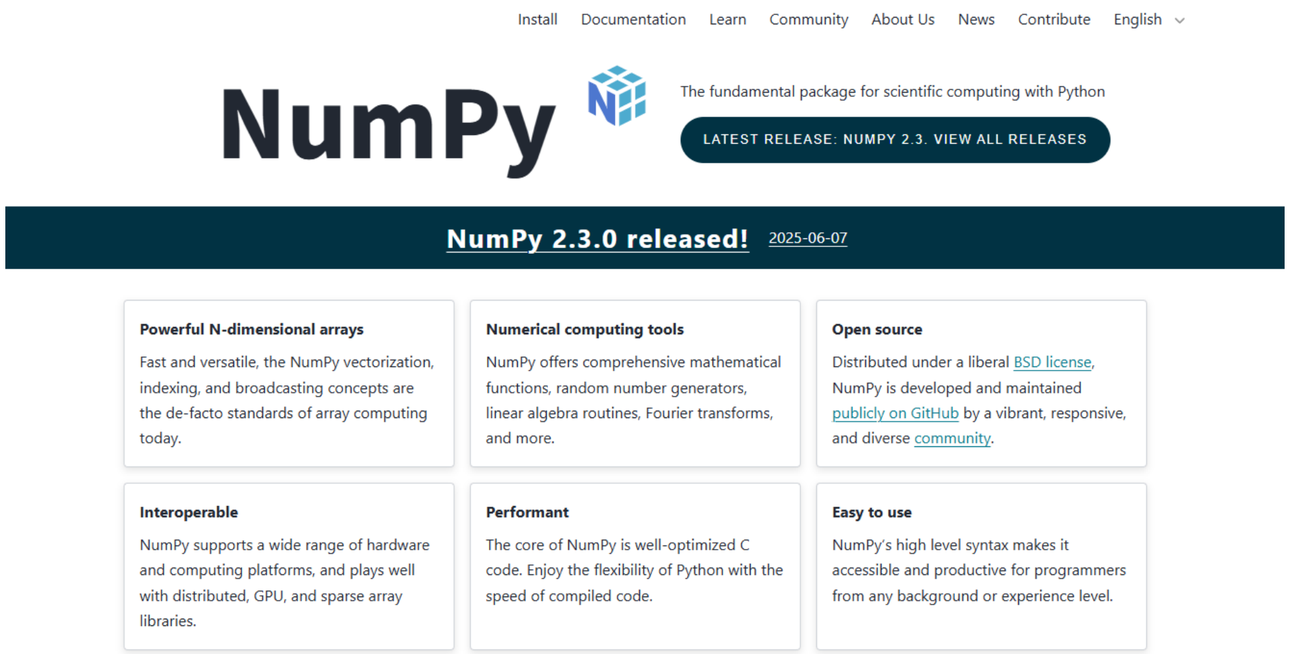
강의개요

- NumPy
  - NumPy 개요
  - NumPy 설치와 임포트
- **다차원 배열 객체**
  - 다차원 배열 생성
  - 배열 산술 연산
  - 배열 인덱싱
  - 배열 슬라이싱
  - 배열 수정
  - 배열 연결
  - 배열 분할
- **유니버설 함수**
  - 유니버설 함수 개요
  - 단항 유니버설 함수
  - 이항 유니버설 함수
- **브로드캐스팅**
  - 브로드캐스팅 개요
  - 브로드캐스팅 특징

NumPy

NumPy 개요

- NumPy(Numerical Python) Python에서 수치 계산을 위한 가장 중요한 패키지임
  - 과학 계산을 위한 대부분의 패키지는 NumPy 배열 객체를 데이터 교환을 위한 공통 언어처럼 사용함
  - NumPy 자체는 모델링이나 과학 계산 기능을 제공하지 않음 → 대신 배열 연산에 대한 이해를 바탕으로 Pandas 같은 고수준 도구를 활용할 수 있음



출처: https://numpy.org/

NumPy 개요

- NumPy가 Python 수치 계산 영역에서 중요한 이유 중 하나는 대용량 데이터 배열을 효율적으로 다룰 수 있도록 설계되었기 때문임
  - 예를 들어, 백만 개의 정수를 저장하는 경우 NumPy 배열과 Python 리스트의 성능 차이를 비교할 수 있음

```python
import numpy as np
import time

# NumPy 배열 연산
my_arr = np.arange(1_000_000)
start = time.time()
for _ in range(10): my_arr2 = my_arr * 2
end = time.time()
print("NumPy 배열 실행 시간:", (end - start) * 1000, "ms")

# Python 리스트 연산
my_list = list(range(1_000_000))
start = time.time()
for _ in range(10): my_list2 = [x * 2 for x in my_list]
end = time.time()
print("리스트 실행 시간:", (end - start) * 1000, "ms")
```

Python

```
NumPy 배열 실행 시간: 18.69058609008789 ms
리스트 실행 시간: 450.21891593933105 ms
```

NumPy 설치와 임포트

- NumPy는 기본적으로 pip 명령어를 통해 설치할 수 있음
  - 새로운 가상환경(venv, conda 등)을 생성한 경우에는 NumPy를 설치하는 것이 권장됨
  - 설치 명령어 예시:

```
pip install numpy

# conda 환경을 사용할 경우
conda install numpy
```

Python

- Python 코드에서 NumPy를 불러올 때는 일반적으로 np라는 별칭을 사용함
  - 관례적으로 import numpy as np를 사용하며, 이는 대부분의 문서와 예제 코드에서 통일된 표기임

```
import numpy as np
print("NumPy 버전:", np.__version__)
```

Python

```
NumPy 버전: 2.1.3
```

다차원 배열 객체

다차원 배열 생성

- NumPy의 핵심 기능 중 하나는 ndarray라 불리는 N차원 배열 객체이며, Python에서 대규모 데이터를 빠르고 유연하게 다룰 수 있는 자료구조임
  - 다차원 배열을 생성하는 가장 쉬운 방법은 NumPy 패키지의 array( ) 함수를 이용하는 것임
  - Python 리스트와 달리 NumPy 배열의 모든 요소는 동일한 데이터 타입이어야 하며, 타입이 섞일 경우 가능한 상위 타입으로 자동 변환됨

```python
# 1차원 배열 생성
arr_1d = np.array([1, 2, 3, 4])
print("1차원 배열:\n", arr_1d)

# 2차원 배열 생성
arr_2d = np.array([[1, 2, 3, 4],
                   [5, 6, 7, 8]])
print("\n2차원 배열:\n", arr_2d)
```

Python

```
1차원 배열:
 [1 2 3 4]

2차원 배열:
 [[1 2 3 4]
 [5 6 7 8]]
```

다차원 배열 생성

- NumPy 배열은 각 속성을 통해 배열 구조와 데이터 타입을 직관적으로 확인할 수 있음
  - ndim: 배열의 차원 개수
  - shape: 각 차원의 크기를 튜플 형태로 반환
  - size: 전체 배열 원소 개수
  - dtype: 배열 원소의 데이터 타입

```python
# 배열 속성 출력
print("차원 수 (ndim):", arr_2d.ndim)
print("배열 형태 (shape):", arr_2d.shape)
print("원소 개수 (size):", arr_2d.size)
print("데이터 타입 (dtype):", arr_2d.dtype)
```

Python

```
차원 수 (ndim): 2
배열 형태 (shape): (2, 4)
원소 개수 (size): 8
데이터 타입 (dtype): int64
```

다차원 배열 생성

- 특히 규모가 큰 배열의 경우에는 NumPy에 내장된 다양한 함수를 사용해 배열을 생성하는 것이 더 효율적일 수 있음
  - 예를 들어, zeros와 ones는 주어진 길이나 모양에 각각 0과 1이 들어 있는 배열을 생성함
  - NumPy는 산술 연산에 최적화되어 있기 때문에 자료형을 명시하지 않으면 기본적으로 부동소수점(float64)으로 출력됨

```python
# 모든 원소를 0으로 초기화
arr1 = np.zeros(10)
print("zeros((3, 6):\n", arr1)

# 모든 원소를 1로 초기화
arr2 = np.ones((3, 6))
print("\nones((3, 6)):\n", arr2)
```

Python

```
zeros((3, 6):
 [0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]

ones((3, 6)):
 [[1. 1. 1. 1. 1. 1.]
 [1. 1. 1. 1. 1. 1.]
 [1. 1. 1. 1. 1. 1.]]
```

다차원 배열 생성

- 표준 배열 생성 함수

함수	설명
array	입력 데이터(리스트, 튜플, 배열 또는 다른 순차형 데이터)를 ndarray로 변환하며, dtype을 명시하지 않은 경우 자료형을 추론하여 저장함
arange	내장 range 함수와 유사하지만 리스트 대신 ndarray를 반환함
ones	주어진 dtype과 모양을 가지는 배열을 생성하고, 내용을 모두 1로 초기화함
zeros	ones와 동일하지만 내용을 0으로 채움
full	인자로 받은 dtype과 배열의 모양을 가지는 배열을 생성하고, 인자로 받은 값으로 배열을 채움
linspace	인자로 받은 값 사이에 일정한 간격을 가지는 배열을 생성함
eye, identity	N × N 크기의 단위행렬을 생성함

다차원 배열 생성

- Python에서 numpy.random 함수는 내장 random 함수를 보강하여 다양한 종류의 확률분포로부터 효율적으로 표본값을 생성하는 데 주로 사용함
  - 예를 들어, normal 함수를 사용하면 표준정규분포로부터 4 × 4 크기의 표본을 생성할 수 있음

```python
# 표준정규분포 난수 배열 생성
samples = np.random.normal(size=(4, 4))
print(samples)
```

Python

```
[[-0.3541278  -0.29953835  0.19651161  0.51243895]
 [-0.15076789 -0.11459721  0.96742204 -1.5494639 ]
 [-0.34275429 -0.80395325  0.68339384  0.82695454]
 [ 0.60282623  0.88248977  1.23086227  1.40117686]]
```

다차원 배열 생성

- numy.random 함수

함수	설명
seed	난수 생성기의 시드를 지정함
shuffle	리스트나 배열의 순서를 뒤섞음
rand	균등분포에서 표본을 추출함
randint	주어진 최소/최대 범위 안에서 임의의 정수를 추출함
randn	평균이 0이고 표준편차가 1인 정규분포에서 표본을 추출함
normal	정규분포(가우시안)에서 표본을 추출함

배열 산술 연산

- 배열의 중요한 특징은 반복문을 사용하지 않고 데이터를 일괄 처리할 수 있음
  - 이를 벡터화(vectorization)라고 하며, 같은 크기의 배열 간 산술 연산은 배열의 각 원소 단위로 적용됨

```python
# 0 이상 20 미만의 정수를 원소로 갖는 2x3 배열 생성
arr = np.random.randint(20, size=(2, 3))
print("원본 배열:\n", arr)

# 배열 곱셈 (각 원소 제곱)
print("\narr * arr (원소별 곱):\n", arr * arr)

# 배열 뺄셈 (자기 자신과의 차이 → 0 배열)
print("\narr - arr (원소별 차):\n", arr - arr)
```

Python

```
원본 배열:
 [[ 6 13 19]
 [16  6 14]]

arr * arr (원소별 곱):
 [[ 36 169 361]
 [256  36 196]]

arr - arr (원소별 차):
 [[0 0 0]
 [0 0 0]]
```

배열 산술 연산

- 스칼라 인자가 포함된 산술 연산의 경우에는 배열 내의 모든 원소에 스칼라 인자가 적용됨
  - 크기가 다른 배열 간의 연산은 브로드캐스팅(broadcasting)이라고 하며, 다음 장에서 자세히 설명함

```python
# 스칼라 나눗셈
print("1 / arr (각 원소의 역수):\n", 1 / arr)

# 스칼라 곱셈
print("\narr * 0.5 (각 원소를 절반으로 줄임):\n", arr * 0.5)
```

Python

```
1 / arr (각 원소의 역수):
 [[0.11111111 0.05555556 0.06666667]
 [0.08333333 0.5        0.05555556]]

arr * 0.5 (각 원소를 절반으로 줄임):
 [[4.5 9.  7.5]
 [6.  1.  9. ]]
```

배열 산술 연산

- 같은 크기를 가지는 배열 간의 비교 연산은 논리값 배열을 반환함

```python
# 0 이상 20 미만의 정수를 원소로 갖는 2x3 배열 생성
arr1 = np.random.randint(20, size=(2, 3))
print("배열 arr1:\n", arr1)

arr2 = np.random.randint(20, size=(2, 3))
print("\n배열 arr2:\n", arr2)

# 두 배열을 비교: arr2 > arr1
print("\narr2 > arr1 비교 결과:\n", arr2 > arr1)
```

Python

```
배열 arr1:
 [[10  5  0]
 [11  2 10]]

배열 arr2:
 [[13 18  4]
 [15 11 12]]

arr2 > arr1 비교 결과:
 [[ True  True  True]
 [ True  True  True]]
```

배열 인덱싱

- NumPy 인덱싱에서 1차원 배열은 표면적으로 Python의 리스트와 유사함
  - 1차원 배열의 특정 위치 값에 접근하려면 대괄호 안에 위치 값을 지정하면 됨

```python
# 0 이상 10 미만의 정수를 원소로 갖는 길이 6의 배열 생성
x1 = np.random.randint(10, size=6)
print("배열 x1:", x1)

# 첫 번째 원소 접근
print("x1[0] =", x1[0])

# 여섯 번째 원소 접근 (인덱스 5)
print("x1[5] =", x1[5])

# 마지막 원소 접근 (음수 인덱스 사용)
print("x1[-1] =", x1[-1])
```

Python

```
배열 x1: [0 0 9 3 4 0]
x1[0] = 0
x1[5] = 0
x1[-1] = 0
```

배열 인덱싱

- 다차원 배열에서는 콤마(,)로 구분된 인덱스 값을 이용해 특정 위치 값에 접근함
  - 아래 예제와 같이 다차원 배열에서 특정 위치 값을 수정할 수도 있음

```python
# 0 이상 10 미만의 정수를 원소로 갖는 (2x4) 배열 생성
x2 = np.random.randint(10, size=(2, 4))
print("배열 x2:\n", x2)

# 특정 원소 접근 (0행 0열 원소)
print("\nx2[0, 0] =", x2[0, 0])

# 특정 원소 값 수정 (0행 0열 원소를 12로 변경)
x2[0, 0] = 12
print("\n값 수정 후 배열 x2:")
print(x2)
```

Python

```
배열 x2:
 [[2 2 6 1]
 [3 9 6 1]]

x2[0, 0] = 2

값 수정 후 배열 x2:
[[12  2  6  1]
 [ 3  9  6  1]]
```

배열 슬라이싱

- NumPy 슬라이싱 구문은 표준 Python의 슬라이싱 구문을 따름
  - 구문에 값이 지정되지 않으면 기본값으로 start=0, stop=차원 크기, step=1이 설정됨

```python
# 0부터 9까지의 정수를 원소로 갖는 배열 생성
x = np.arange(10)
print("배열 x:", x)

# 인덱스 5부터 끝까지 선택
print("\nx[5:]:", x[5:])

# 인덱스 4 이상 7 미만 선택
print("x[4:7]:", x[4:7])

# 처음부터 끝까지 2칸씩 건너뛰어 선택
print("x[::2]:", x[::2])

# 인덱스 5부터 시작하여 역순으로 2칸씩 건너뛰어 선택
print("x[5::-2]:", x[5::-2])
```

Python

```
배열 x: [0 1 2 3 4 5 6 7 8 9]

x[5:]: [5 6 7 8 9]
x[4:7]: [4 5 6]
x[::2]: [0 2 4 6 8]
x[5::-2]: [5 3 1]
```

배열 수정

- 배열의 구조를 변경하고자 할 때 가장 유연하게 하는 방법은 reshape( ) 함수를 사용하는 것임

```python
# 0부터 11까지의 정수를 원소로 갖는 배열 생성
x = np.arange(12)
print("배열 x:", x)

# 1차원 배열을 (3x4) 2차원 배열로 변환
print("\nx.reshape(3, 4):\n", x.reshape(2, 6))

# 1차원 배열을 (3x2x2) 3차원 배열로 변환
print("\nx.reshape(3, 2, 2):\n", x.reshape(2, 2, 3))
```

Python

```
배열 x: [ 0  1  2  3  4  5  6  7  8  9 10 11]

x.reshape(3, 4):
 [[ 0  1  2  3  4  5]
 [ 6  7  8  9 10 11]]

x.reshape(3, 2, 2):
 [[[ 0  1  2]
  [ 3  4  5]]

 [[ 6  7  8]
  [ 9 10 11]]]
```

배열 수정

- 배열 전치는 데이터를 복사하지 않고 데이터의 모양만 바꾼 뷰(view)를 반환하는 특별한 기능임
  - ndarray는 transpose( ) 메서드와 T 속성을 제공함

```python
# 0부터 14까지의 정수를 (3x5) 배열로 생성
arr = np.arange(15).reshape(3, 5)
print("원본 배열 arr:\n", arr)

# 배열 전치 (행과 열을 교환)
print("\narr.T:\n", arr.T)
```

Python

```
원본 배열 arr:
 [[ 0  1  2  3  4]
 [ 5  6  7  8  9]
 [10 11 12 13 14]]

arr.T:
 [[ 0  5 10]
 [ 1  6 11]
 [ 2  7 12]
 [ 3  8 13]
 [ 4  9 14]]
```

배열 연결

- NumPy는 여러 배열을 하나로 결합하거나, 반대로 하나의 배열을 여러 배열로 분할 가능함
  - np.concatenate 함수는 튜플이나 리스트 형태의 배열을 첫 번째 인자로 지정해야 함
  - axis 인자를 통해 어떤 축을 기준으로 연결할지 결정할 수 있음

```python
# 0 이상 6 미만의 정수를 원소로 갖는 (2x3) 배열 생성
x = np.random.randint(6, size=(2, 3))  # 2x3 배열
y = np.random.randint(6, size=(2, 3))  # 2x3 배열

# 행 방향(세로)으로 연결
print("np.concatenate([x, y]):\n", np.concatenate([x, y]))

# 열 방향(가로)으로 연결
print("\nnp.concatenate([x, y], axis=1):\n", np.concatenate([x, y], axis=1))
```

Python

```
np.concatenate([x, y]):
 [[1 3 1]
 [4 3 1]
 [5 2 3]
 [4 3 1]]

np.concatenate([x, y], axis=1):
 [[1 3 1 5 2 3]
 [4 3 1 4 3 1]]
```

배열 연결

- np.vstack, np.hstack은 차원이 혼합된 경우 더 직관적이고 명확하게 연결할 수 있음

```python
# 세로 방향으로 쌓기
print("np.vstack([x, y]):\n", np.vstack([x, y]))

# 가로 방향으로 쌓기
print("\nnp.hstack([x, y]):\n", np.hstack([x, y]))
```

Python

```
np.vstack([x, y]):
 [[1 3 1]
 [4 3 1]
 [5 2 3]
 [4 3 1]]

np.hstack([x, y]):
 [[1 3 1 5 2 3]
 [4 3 1 4 3 1]]
```

배열 분할

- np.split은 범용적인 배열 분할 함수이며, 축(axis)을 지정하여 원하는 위치에서 배열을 나눌 수 있음

```python
# 0~15까지 숫자를 담은 4x4 배열 생성
z = np.arange(16).reshape(4, 4)
print("원본 배열 z:\n", z)

# 2행을 기준으로 위/아래 분할 (axis=0)
upper, lower = np.split(z, [2], axis=0)
print("\n위쪽 배열:\n", upper)
print("\n아래쪽 배열:\n", lower)
```

Python

```
원본 배열 z:
 [[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]
 [12 13 14 15]]

위쪽 배열:
 [[0 1 2 3]
 [4 5 6 7]]

아래쪽 배열:
 [[ 8  9 10 11]
 [12 13 14 15]]
```

배열 분할

- np.split을 사용할 수도 있지만, 전용 함수를 쓰면 가독성이 높아짐
  - np.vsplit: 수직(vertical) 분할하며, 행 단위로 배열을 나눔 (axis=0)
  - Np.hsplit: 수평(horizontal) 분할하며, 열 단위로 배열을 나눔 (axis=1)

```python
# 수평 분할 (열 기준으로 좌/우 나누기)
left, right = np.hsplit(z, [2])
print("왼쪽 배열:\n", left)
print("\n오른쪽 배열:\n", right)
```

Python

```
왼쪽 배열:
 [[ 0  1]
 [ 4  5]
 [ 8  9]
 [12 13]]

오른쪽 배열:
 [[ 2  3]
 [ 6  7]
 [10 11]
 [14 15]]
```

유니버설 함수

유니버설 함수 개요

- 유니버설 함수(Universal Functions, UFuncs)는 ndarray의 각 원소 단위로 연산을 수행하는 함수임
  - 반복 연산을 벡터화(vectorization)하여 빠른 계산이 가능함
  - 단항 유니버설 함수: 하나의 인자를 받음 (np.abs, np.sqrt 등)
  - 이항 유니버설 함수: 두 개의 인자를 받아 배열 반환 (np.add, np.maximum 등)

```python
# 배열 생성
x = np.array([1, 2, 3])
y = np.array([10, 5, 7])

# 단항 유니버설 함수
print("단항 유니버설 함수:", np.abs(x))

# 이항 유니버설 함수
print("이항 유니버설 함수:", np.add(x, y))
```

Python

```
단항 유니버설 함수: [1 2 3]
이항 유니버설 함수: [11  7 10]
```

단항 유니버설 함수

- 각 원소별로 단항 유니버설 함수를 적용할 수 있음

```python
# 배열 생성
arr = np.array([1, 4, 9, 16])
print("원본 배열 arr:", arr)

# 제곱근
print("\nnp.sqrt(arr): \n", np.sqrt(arr))

# 지수 함수
print("\nnp.exp(arr): \n", np.exp(arr))
```

Python

```
원본 배열 arr: [ 1  4  9 16]

np.sqrt(arr):
 [1. 2. 3. 4.]

np.exp(arr):
 [2.71828183e+00 5.45981500e+01 8.10308393e+03 8.88611052e+06]
```

이항 유니버설 함수

- 두 배열의 원소 간 연산을 자동으로 수행함

```python
# 배열 생성
x = np.array([1, 2, 3])
y = np.array([10, 20, 30])

# 배열 덧셈
print("np.add(x, y): \n", np.add(x, y))

# 두 값 중 큰 값
print("np.maximum(x, y):\n", np.maximum(x, y))
```

Python

```
np.add(x, y):
 [11 22 33]
np.maximum(x, y):
 [10 20 30]
```

브로드캐스팅

브로드캐스팅 개요

- 브로드캐스팅(Broadcasting)은 서로 다른 크기의 배열에 이항 유니버설 함수(덧셈, 뺄셈, 곱셈 등)를 적용하기 위한 규칙의 집합임
  - 같은 크기의 배열에서는 이항 연산이 원소 단위로 수행됨
  - 크기가 다른 배열 간에도 브로드캐스팅 규칙에 따라 연산이 가능함
  - 작은 배열이 큰 배열의 크기에 맞게 자동으로 확장된다고 이해할 수 있음

```python
# 배열 생성
a = np.array([[1, 2, 3],
              [4, 5, 6]])
b = np.array([10, 20, 30])

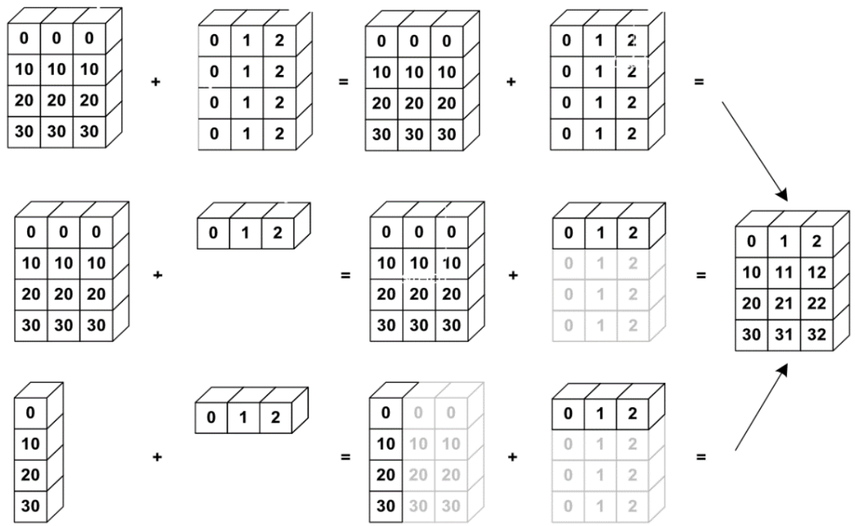
# 브로드캐스팅을 통한 배열 덧셈
print(a + b)
```

Python

```
[[11 22 33]
 [14 25 36]]
```

브로드캐스팅 특징

- 브로드캐스팅은 두 배열 사이의 상호작용을 결정하기 위해 엄격한 규칙을 따름
  - 두 배열의 차원 수가 다르면 작은 배열의 앞쪽(왼쪽)을 1로 채움
  - 두 배열의 형상이 어떤 차원에서 일치하지 않으면, 해당 차원의 형상이 1인 배열을 다른 형상과 일치하도록 늘림
  - 임의의 차원에서 크기가 다르고 1도 아니라면 오류 발생



브로드캐스팅 특징

- 1차원 배열에 2차원 배열을 더하는 경우, 작은 배열이 자동으로 확장되어 계산됨

```python
# (2, 3) 2차원 배열 생성
arr1 = np.ones((2, 3))
# (3,) 1차원 배열 생성
arr2 = np.arange(3)

print("arr1:\n", arr1)
print("\narr2:\n", arr2)

# arr2가 자동으로 (1, 3) → (2, 3)으로 확장되어 arr1과 더해짐
print("\narr1 + arr2:\n", arr1 + arr2)
```

Python

```
arr1:
 [[1. 1. 1.]
 [1. 1. 1.]]

arr2:
 [0 1 2]

arr1 + arr2:
 [[1. 2. 3.]
 [1. 2. 3.]]
```

브로드캐스팅 특징

- 두 배열 모두 브로드캐스팅이 필요할 때, 두 배열이 각 축을 따라 자동 확장되어 연산 수행됨

```python
# (3, 1) 2차원 배열
arr1 = np.arange(3).reshape((3, 1))
# (3,) 1차원 배열
arr2 = np.arange(3)

print("arr1:\n", arr1)
print("\narr2:\n", arr2)

# arr1: (3,1) → (3,3)으로 확장
# arr2: (3,)  → (1,3) → (3,3)으로 확장
print("\narr1 + arr2:\n", arr1 + arr2)
```

Python

```
arr1:
 [[0]
 [1]
 [2]]

arr2:
 [0 1 2]

arr1 + arr2:
 [[0 1 2]
 [1 2 3]
 [2 3 4]]
```

브로드캐스팅 특징

- 브로드캐스팅 규칙을 만족하지 않으면 연산 불가하며, ValueError 발생함

```python
# (3, 2) 배열
arr1 = np.ones((3, 2))
# (3,) 모양의 배열
arr2 = np.arange(3)

print("arr1:\n", arr1)
print("\narr2:\n", arr2)

# 마지막 차원 비교 → 2 ≠ 3, 둘 다 1이 아님 → 브로드캐스팅 불가
print("\narr1 + arr2:", arr1 + arr2)
```

Python

```
arr1:
 [[1. 1.]
 [1. 1.]
 [1. 1.]]

arr2:
 [0 1 2]

---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[59], line 10
      7 print("\narr2:\n", arr2)
      9 # 마지막 차원 비교 → 2 ≠ 3, 둘 다 1이 아님 → 브로드캐스팅 불가
---> 10 print("\narr1 + arr2:", arr1 + arr2)

ValueError: operands could not be broadcast together with shapes (3,2) (3,)
```

수고하셨습니다.